A PROJECT REPORT

on

TRAFFIC SIGN RECOGNITION FOR AUTONOMOUS VEHICLES

SignSense: A Deep Learning System with a FastAPI Service and a Streamlit Driving Simulation

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING

Submitted by

Vaibhav Anand Singh

Roll No.: BTECH/10261/23

7th Semester, Session 2026-27 (Monsoon 2026)

Under the guidance of

Dr. Jamimamul Bakas

Department of Computer Science and Engineering

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BIRLA INSTITUTE OF TECHNOLOGY, MESRA

RANCHI, JHARKHAND 835215, INDIA

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to my project guide, **Dr. Jamimamul Bakas**, Department of Computer Science and Engineering, Birla Institute of Technology, Mesra, for the guidance, encouragement and valuable feedback provided throughout this project. The suggestions on how to frame the problem, evaluate the model honestly and present the results helped shape this work at every stage.

I am thankful to the Head of the Department and all the faculty members of the Department of Computer Science and Engineering for the knowledge they have shared over the course of the programme, and for creating an environment in which students are encouraged to build complete, working systems. I also thank Birla Institute of Technology, Mesra for providing the academic resources and facilities that made this project possible.

I acknowledge the creators of the German Traffic Sign Recognition Benchmark at the Institut für Neuroinformatik, Ruhr-Universität Bochum, for making the dataset publicly available, and the developers of the open-source projects used in this work, in particular PyTorch, torchvision, FastAPI, Streamlit and scikit-learn.

Finally, I am grateful to my family and friends for their constant support, patience and motivation during the course of this project.

Place: Mesra, Ranchi

Date: _____

Vaibhav Anand Singh

Roll No.: BTECH/10261/23

B.Tech (CSE), 7th Semester

ABSTRACT

Traffic sign recognition is a core perception task for autonomous and driver-assisted vehicles: a car that cannot reliably read a speed limit, stop or yield sign cannot plan safe behaviour. This project presents **SignSense**, an end-to-end traffic sign recognition system written entirely in Python. A compact VGG-style convolutional neural network, SignNet, with 1,478,411 parameters, is trained from scratch with PyTorch on the German Traffic Sign Recognition Benchmark (GTSRB), which contains 43 sign classes and 39,270 real-world images. Images are resized to 48×48 pixels, normalised and augmented with random affine and colour transformations; horizontal flipping is deliberately excluded because it changes the meaning of directional signs.

Trained for 15 epochs with the AdamW optimiser and a cosine learning-rate schedule on an Apple M3 GPU (about eight minutes), the model reaches **98.80% accuracy** and a **macro-averaged F1 score of 0.9821** on the 12,630-image official test set, on par with the 98.84% average human accuracy reported for the same benchmark. The trained model is served by a FastAPI backend that validates uploads, returns top-k predictions with confidence scores and maps every class to a driving action. A Streamlit frontend offers single-image and batch classification, a model report and a driving simulation in which a rule-based planner turns predictions into speed commands. Predictions below a confidence threshold of 0.6 are flagged, so the simulated vehicle keeps its current behaviour instead of acting on an uncertain sign.

Single-image inference takes a median of 3.3 ms, the service classified 85 of 86 held-out demo images correctly (the single error was flagged as low confidence), and the code base is covered by 21 automated tests. This report describes the motivation, related work, dataset, system design, implementation, evaluation, limitations and future scope of the project.

Keywords: traffic sign recognition, convolutional neural network, GTSRB, autonomous vehicles, deep learning, PyTorch, FastAPI, Streamlit.

TABLE OF CONTENTS

ACKNOWLEDGEMENT . . . . . **i**

ABSTRACT . . . . . **ii**

LIST OF FIGURES . . . . . **vi**

LIST OF TABLES . . . . . **vii**

LIST OF ABBREVIATIONS . . . . . **viii**

1. Introduction **1**

1.1 Background 1

1.2 Problem Statement 1

1.3 Objectives 1

1.4 Scope of the Project 2

1.5 Organisation of the Report 2

2. Literature Review **3**

2.1 Classical Approaches 3

2.2 The GTSRB Benchmark 3

2.3 Deep Learning Approaches 3

2.4 Research Gap and Motivation 4

3. Requirements and Tools **5**

3.1 Functional Requirements 5

3.2 Non-Functional Requirements 5

3.3 Hardware and Software Environment 5

4. Dataset **7**

4.1 The German Traffic Sign Recognition Benchmark 7

4.2 Data Splits 7

4.3 Class Imbalance 8

4.4 A Note on Validation Leakage . . . 8

5. System Design . . . 9

5.1 Overall Architecture . . . 9

5.2 Project Structure . . . 10

5.3 Life of a Prediction Request . . . 10

5.4 From Sign Class to Vehicle Action . . . 11

6. Methodology and Implementation . . . 12

6.1 Preprocessing . . . 12

6.2 Data Augmentation . . . 12

6.3 SignNet Architecture . . . 12

6.4 Training Procedure . . . 13

6.5 Evaluation Metrics . . . 14

6.6 Backend Implementation . . . 15

6.7 Frontend Implementation . . . 16

7. Results and Discussion . . . 17

7.1 Training Behaviour . . . 17

7.2 Test-Set Performance . . . 18

7.3 Per-Class Analysis . . . 19

7.4 Confusion Analysis . . . 20

7.5 API and Inference Performance . . . 21

7.6 Application Demonstration . . . 22

7.7 Discussion . . . 26

8. Testing and Validation . . . 27

8.1 Automated Tests . . . 27

8.2 System-Level Validation . . . 28

9. Conclusion and Future Scope . . . 29

9.1 Conclusion . . . 29

9.2 Limitations . . . 29

9.3 Future Scope . . . 30

REFERENCES **31**

Appendix A: Source Code Listings **33**

A.1 SignNet model 33

A.2 Data pipeline and augmentation 33

A.3 Training loop 34

A.4 Inference wrapper 36

A.5 FastAPI backend 37

A.6 Rule-based planner 39

A.7 Sign semantics 39

Appendix B: GTSRB Class Reference **41**

LIST OF FIGURES

Figure 4.1: One test-set sample for each of the 43 GTSRB classes (class IDs above each image) . . 7

Figure 4.2: Number of training images per class, coloured by sign category 8

Figure 5.1: System architecture of SignSense 9

Figure 6.1: SignNet architecture: feature maps after each stage 13

Figure 7.1: Training and validation loss (a) and accuracy (b) per epoch 17

Figure 7.2: Per-class F1 score on the test set (red: F1 below 0.96) 19

Figure 7.3: Row-normalised confusion matrix on the GTSRB test set (rows: true class, columns: predicted class) 20

Figure 7.4: Classify tab: a Stop sign recognised with 99.9% confidence 23

Figure 7.5: Low-confidence prediction: the action is withheld until the sign is confirmed 23

Figure 7.6: Batch tab: 12 random test samples classified in one request 24

Figure 7.7: Drive simulation: the planner's speed and limit over a 12-sign route 25

Figure 7.8: Model report tab with headline metrics, curves and the confusion matrix 26

LIST OF TABLES

Table 2.1: Reported results on the GTSRB test set 4
Table 3.1: Hardware and software used in the project 6
Table 4.1: Dataset splits used in this project 8
Table 5.1: Sign categories, actions and planner rules 11
Table 6.1: Training-time data augmentation 12
Table 6.2: SignNet layer summary 13
Table 6.3: Training configuration 14
Table 6.4: REST API endpoints 15
Table 7.1: Per-epoch training history (best epoch 12 in bold) 18
Table 7.2: Summary of test-set results 18
Table 7.3: The five weakest classes 19
Table 7.4: Most frequent confusions on the test set 21
Table 7.5: API accuracy and latency on the demo images 22
Table 8.1: Automated test suite 27
Table 8.2: System-level validation scenarios 28
Table B.1: GTSRB classes, categories, image counts and test F1 scores 41

LIST OF ABBREVIATIONS

ADAS	Advanced Driver-Assistance System
API	Application Programming Interface
BN	Batch Normalisation
CNN	Convolutional Neural Network
CORS	Cross-Origin Resource Sharing
CUDA	Compute Unified Device Architecture (NVIDIA)
FC	Fully Connected (layer)
FN / FP / TP	False Negative / False Positive / True Positive
GPU	Graphics Processing Unit
GTSDB	German Traffic Sign Detection Benchmark
GTSRB	German Traffic Sign Recognition Benchmark
HOG	Histogram of Oriented Gradients
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
LR	Learning Rate
MPS	Metal Performance Shaders (Apple GPU backend for PyTorch)
ReLU	Rectified Linear Unit
REST	Representational State Transfer
SVM	Support Vector Machine
UI	User Interface

CHAPTER 1
Introduction

1.1 Background

Autonomous vehicles and advanced driver-assistance systems (ADAS) must perceive their surroundings, decide what to do and then act. Road signs are one of the most important inputs to that decision, because they encode the rules of the road at a particular place: the maximum legal speed, who has priority at a junction, where entry is forbidden and which hazards lie ahead. Human drivers read these signs almost without thinking, but a vehicle has to recognise them from camera images that may be small, blurred, badly lit, partly occluded or seen at an angle.

Traffic sign recognition (TSR) is therefore a standard component of the perception stack of a self-driving car, alongside lane, vehicle and pedestrian detection. Modern TSR systems are built with deep convolutional neural networks (CNNs), which learn visual features directly from labelled images instead of relying on hand-crafted colour and shape rules [1], [3], [13]. Public benchmarks such as the German Traffic Sign Recognition Benchmark (GTSRB) [1], [2] make it possible to compare methods on the same data, and they are widely used in research and in self-driving car simulations.

1.2 Problem Statement

Given a cropped colour image of a traffic sign taken from a vehicle-mounted camera, the system must identify which of 43 sign classes it belongs to, report how confident it is, and translate the result into an action that a simulated autonomous vehicle can follow. The system must be accurate on unseen images, fast enough for interactive use, robust to invalid input, and usable by someone who is not a machine-learning specialist through a simple web interface.

1.3 Objectives

The objectives of this project are:

- To build a reproducible data pipeline for the GTSRB dataset with a held-out validation split and augmentation that respects the meaning of traffic signs.
- To design and train a compact CNN that achieves at least 97% accuracy on the official GTSRB test set while remaining small enough to run in real time on a laptop GPU or CPU.

- To evaluate the model with accuracy, per-class F1 scores and a confusion matrix, and to understand where and why it fails.
- To expose the model through a documented REST API (FastAPI) that validates input, returns top-k predictions with confidence and maps each class to a vehicle action.
- To build an interactive Streamlit frontend with single-image, batch and camera input, a model report and a driving simulation that demonstrates how predictions drive vehicle behaviour.
- To make the system safe by design by flagging low-confidence predictions, and to cover it with automated tests.

1.4 Scope of the Project

The project addresses the *classification* stage of traffic sign recognition: the input is an image that already contains a single sign, as produced by a detector or by manual cropping. Locating signs inside full dash-camera frames (detection) is outside the scope of this minimum viable product and is discussed as future work. The driving behaviour is simulated with a rule-based planner rather than a physical vehicle or a full driving simulator. The model is trained on German signs, which follow the Vienna Convention conventions shared by many countries.

1.5 Organisation of the Report

Chapter 2 reviews classical and deep-learning approaches to traffic sign recognition. Chapter 3 lists the requirements and the hardware and software used. Chapter 4 describes the GTSRB dataset. Chapter 5 presents the overall system design, and Chapter 6 explains the methodology and implementation of the data pipeline, the SignNet model, the training procedure, the backend and the frontend. Chapter 7 reports and discusses the results, Chapter 8 covers testing, and Chapter 9 concludes with limitations and future scope. Appendix A contains key source code listings and Appendix B lists all 43 classes.

CHAPTER 2
Literature Review

2.1 Classical Approaches

Early traffic sign recognition systems relied on the strong visual design of signs. A typical pipeline segmented candidate regions using colour thresholds (red, blue, yellow) in colour spaces such as HSV, filtered them by shape (circles, triangles, octagons), and classified them with hand-crafted features. Histograms of Oriented Gradients (HOG) [14] combined with classifiers such as Support Vector Machines (SVM) or linear discriminant analysis were among the strongest of these methods and served as baselines in the GTSRB competition [1]. These approaches are fast and interpretable, but they are sensitive to lighting, faded paint, motion blur and occlusion, and the features have to be engineered by hand.

2.2 The GTSRB Benchmark

Stallkamp et al. introduced the German Traffic Sign Recognition Benchmark as a multi-class classification competition at IJCNN 2011 [1]. It contains more than 50,000 images of 43 classes captured from video on German roads, with large variation in scale, illumination and viewpoint. In a follow-up study [2] the authors measured human performance on the same test set and reported an average human accuracy of 98.84%, which has since become a reference point for automatic methods. A companion benchmark for detection, GTSDB, was released later [20].

2.3 Deep Learning Approaches

Convolutional neural networks, introduced for document recognition by LeCun et al. [13], dominated the GTSRB competition. Cireşan et al. won with a multi-column deep neural network, a committee of CNNs trained on differently pre-processed images, reaching 99.46% accuracy and exceeding human performance [3]. Sermanet and LeCun used a multi-scale CNN in which features from several layers are fed to the classifier, achieving 98.31% in the competition [4], [2]. Later work added spatial transformer networks [11], which learn to undo rotations and scale changes inside the network; Arcos-García et al. reported 99.71% with such a model [10].

Several general-purpose techniques are central to these results and are used in this project. Deep stacks of small 3×3 convolutions, popularised by the VGG networks [5], give large receptive fields with few parameters. Batch normalisation [6] stabilises and speeds up

training, and dropout [7] reduces overfitting. AdamW [8] decouples weight decay from the adaptive gradient update, and cosine annealing of the learning rate [9] provides a smooth schedule that usually improves final accuracy.

Table 2.1: Reported results on the GTSRB test set

Method	Reference	GTSRB test accuracy
Average human performance	Stallkamp et al., 2012 [2]	98.84%
Multi-scale CNN	Sermanet and LeCun, 2011 [4]	98.31%
Multi-column DNN (committee of CNNs)	Cireşan et al., 2012 [3]	99.46%
CNN with spatial transformers	Arcos-García et al., 2018 [10]	99.71%
SignNet, single model, 1.48 M parameters	**This work**	**98.80%**

2.4 Research Gap and Motivation

Most published work reports a single accuracy number for a model evaluated offline. For a student project aimed at self-driving car simulations, two further questions matter: how the model behaves when it is wrong, and how its output can be used by the rest of a vehicle software stack. This project therefore focuses on a compact model that is trained in minutes on a laptop, on a careful error analysis, and on wrapping the model in a service with explicit confidence handling and a simple planner, so the complete path from camera image to vehicle decision can be demonstrated end to end.

CHAPTER 3
Requirements and Tools

3.1 Functional Requirements

- **FR1.** Download and prepare GTSRB automatically, with a reproducible training/validation split.
- **FR2.** Train a CNN classifier and save the best checkpoint with its preprocessing metadata.
- **FR3.** Evaluate the model on the official test set and produce metrics and plots.
- **FR4.** Classify a single image or a batch of up to 32 images through an HTTP API, returning the label, confidence, top-k alternatives, sign category and recommended vehicle action.
- **FR5.** Provide a web interface for upload, test samples and camera input, batch results, a driving simulation and a model report.
- **FR6.** Flag predictions below a configurable confidence threshold as uncertain.

3.2 Non-Functional Requirements

- **Performance:** single-image inference well below 50 ms on a laptop GPU, so the simulation runs smoothly.
- **Robustness:** invalid, empty or oversized uploads must be rejected with clear error messages instead of crashing the service.
- **Portability:** the same code must run on CUDA, Apple MPS or CPU, selected automatically or through the `DEVICE` environment variable.
- **Reproducibility:** fixed random seeds, pinned dependency ranges and saved training history.
- **Maintainability:** a small, modular code base with separate packages for machine learning, the backend and the frontend, covered by automated tests.

3.3 Hardware and Software Environment

Table 3.1: Hardware and software used in the project

Component	Details
Machine	Apple MacBook with Apple M3 chip, 8 GB unified memory
Accelerator	Apple GPU through the PyTorch MPS backend (CUDA and CPU also supported)
Operating system	macOS
Language	Python 3.13
Deep learning	PyTorch 2.15, torchvision 0.29
Data and metrics	NumPy 2.5, pandas 3.1, scikit-learn 1.9, Matplotlib 3.11, Pillow 13
Backend	FastAPI 0.162, Uvicorn 0.54, Pydantic 2.14, python-multipart
Frontend	Streamlit 1.67, Requests 2.34
Testing	pytest 9.2 with FastAPI TestClient
Version control	Git and GitHub (https://github.com/Vaibhavanandsingh/sign_sense)

CHAPTER 4
Dataset

4.1 The German Traffic Sign Recognition Benchmark

GTSRB [1] consists of images of 43 classes of German road signs extracted from video recorded while driving on real roads. Each image contains exactly one sign with a small margin around it, and image sizes range from about 15×15 to 250×250 pixels. The classes cover speed limits, prohibitions, warnings, mandatory directions, priority signs and the ends of restrictions. The dataset is loaded with `torchvision.datasets.GTSRB`, which downloads and verifies the archives on first use. The training archive distributed through torchvision contains 26,640 images, and the official test set contains 12,630 images.

Figure 4.1 shows one test image for each class; the class names are listed in Appendix B. The images illustrate the main difficulties of the task: low resolution, motion blur, very dark or over-exposed frames, and pairs of classes that differ only in a small pictogram or a digit.



Figure 4.1: *One test-set sample for each of the 43 GTSRB classes (class IDs above each image)*

4.2 Data Splits

The official training set is divided at random into training and validation subsets in a 90:10 ratio with a fixed seed (42), and the official test set is used only for the final evaluation.

Table 4.1: Dataset splits used in this project

Split	Images	Share	Purpose
Training	23,976	61.1%	Fitting model weights
Validation	2,664	6.8%	Model selection (best epoch)
Test (official)	12,630	32.2%	Final, unbiased evaluation
Total	**39,270**	**100%**	

4.3 Class Imbalance

The classes are strongly imbalanced (Figure 4.2). The largest classes have 1,500 training images (classes 1, 2: Speed limit (30km/h), Speed limit (50km/h)), while the smallest have only 150 (classes 0, 19, 37: Speed limit (20km/h), Dangerous curve to the left, Go straight or left), a ratio of 10:1. Common signs such as speed limits and priority signs dominate, while many warning signs are rare. The test set has a similar distribution, so overall accuracy is dominated by the frequent classes; this is why the macro-averaged F1 score, which weights every class equally, is reported alongside accuracy.

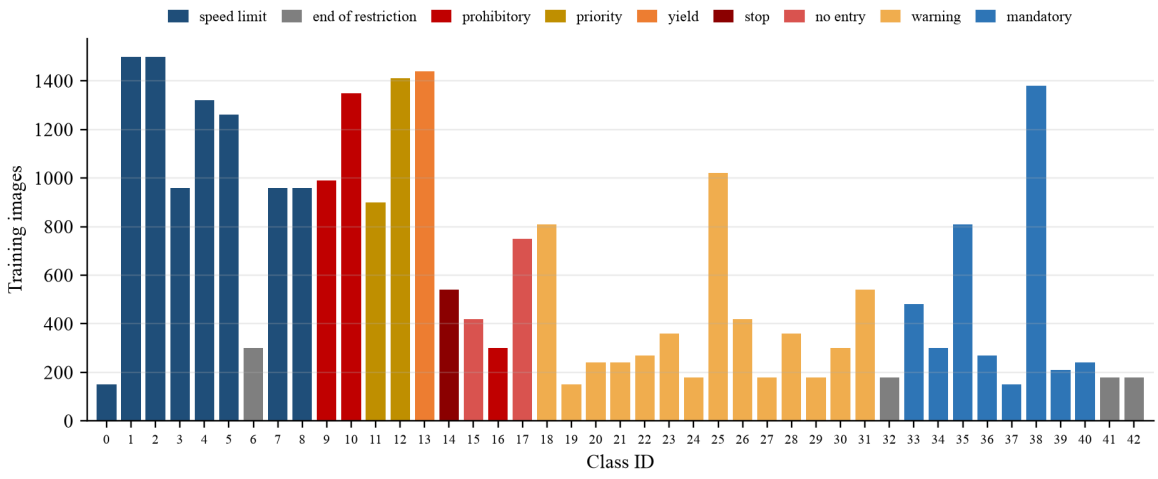


Figure 4.2: *Number of training images per class, coloured by sign category*

4.4 A Note on Validation Leakage

GTSRB training images are organised in *tracks*: about 30 consecutive frames of the same physical sign as the vehicle approaches it. A random split places frames of the same track in both the training and the validation subset, so the validation images are near-duplicates of training images. The validation accuracy is therefore optimistic and is used only to choose the best epoch. All reported performance figures come from the official test set, which was recorded separately.

CHAPTER 5
System Design

5.1 Overall Architecture

The system has two halves (Figure 5.1). The **offline** half, in the `ml` package, prepares the data, trains SignNet and evaluates it, producing a checkpoint file, metrics and plots. The **online** half serves the trained model: a FastAPI backend loads the checkpoint once at start-up and answers HTTP requests, and a Streamlit frontend sends images to the API and presents the results. The two halves share only the checkpoint and a small amount of configuration, so the model can be retrained without touching the service code.

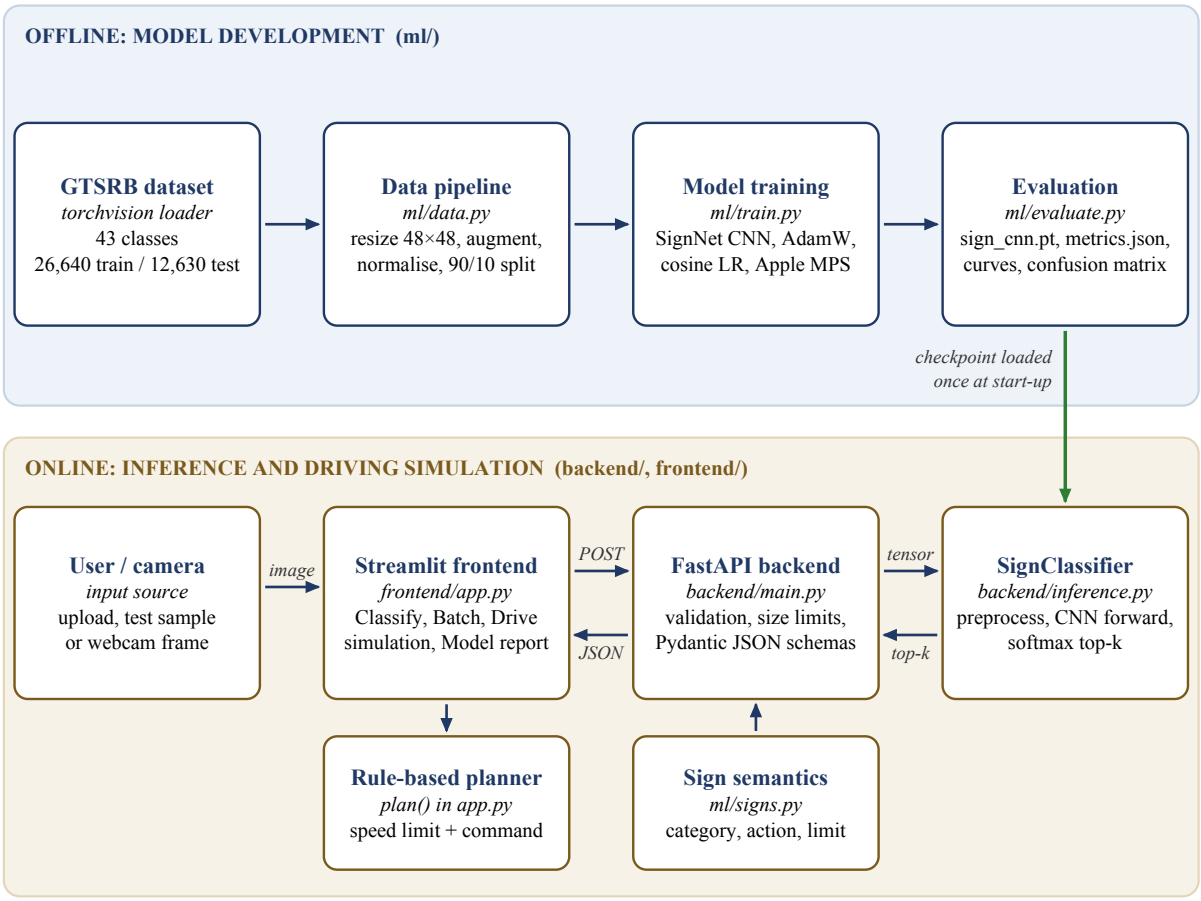


Figure 5.1: *System architecture of SignSense*

5.2 Project Structure

The repository is organised into three Python packages and supporting folders:

```
sign_sense/
|-- ml/                    # data, model, training and evaluation
|   |-- config.py          # paths, image size, normalisation, 43 class names
|   |-- data.py            # transforms and data loaders
|   |-- model.py           # SignNet CNN
|   |-- train.py           # training loop, checkpointing, curves
|   |-- evaluate.py        # test metrics and confusion matrix
|   |-- signs.py           # class -> category, action, speed limit
|   `-- export_samples.py  # stratified demo images
|-- backend/
|   |-- main.py            # FastAPI app and endpoints
|   |-- inference.py       # SignClassifier (load, preprocess, predict)
|   `-- schemas.py         # Pydantic response models
|-- frontend/app.py        # Streamlit user interface and planner
|-- tests/                 # pytest suite (API and ML)
|-- models/sign_cnn.pt     # trained checkpoint
|-- reports/               # metrics.json, history.json, plots
`-- data/samples/          # 86 demo images, 2 per class
```

5.3 Life of a Prediction Request

1. The user uploads an image, picks a test sample or takes a camera frame in the Streamlit app.
2. The frontend sends the image as a multipart form to `POST /predict` with the requested number of alternatives (top-k).
3. FastAPI checks that the file is not empty, is at most 5 MB and can be decoded as an image; otherwise it returns HTTP 400 or 413 with an explanation.
4. The SignClassifier resizes the image to 48×48, normalises it with the training statistics and runs the CNN in a worker thread, so the event loop is never blocked.
5. A softmax turns the 43 logits into probabilities; the top-k classes and their confidences are returned.
6. The backend adds the sign semantics from `ml/signs.py` (category, action and speed limit) and an `is_confident` flag (confidence of at least 0.6), and returns a JSON document.
7. The frontend displays the result and, in the driving simulation, passes it to the planner, which updates the speed limit and the commanded speed.

5.4 From Sign Class to Vehicle Action

Each of the 43 classes is mapped to one of nine categories (Table 5.1). The category determines the action text returned by the API and how the planner in the simulation reacts. Speed limit signs set a new limit; the two end-of-limit signs (classes 6 and 32) restore an open-road limit of 100 km/h; stop and no-entry signs bring the vehicle to a halt; and yield and warning signs scale the speed down.

Table 5.1: Sign categories, actions and planner rules

Category	Class IDs	Action returned by the API	Planner effect
speed_limit	0–5, 7, 8	Set target speed to N km/h	limit = N
end_of_restriction	6, 32, 41, 42	Restore default behaviour	6, 32: limit = 100 km/h
stop	14	Come to a full stop, proceed when clear	speed = 0
no_entry	15, 17	Do not enter, plan an alternative route	speed = 0
yield	13	Slow down and give way	speed = 0.5 × limit
warning	18–31	Hazard ahead, reduce speed	speed = 0.7 × limit
priority	11, 12	Maintain speed, you have right of way	speed = limit
prohibitory	9, 10, 16	Restriction active	speed = limit
mandatory	33–40	Follow mandatory direction	speed = limit
(confidence < 0.6)	any	Keep current behaviour until confirmed	unchanged

CHAPTER 6
Methodology and Implementation

6.1 Preprocessing

Every image is converted to RGB, resized to 48×48 pixels and converted to a tensor with values in [0, 1]. Each channel is then standardised with the per-channel mean (0.3403, 0.3121, 0.3214) and standard deviation (0.2724, 0.2608, 0.2669) of the GTSRB training set. The same values are stored inside the checkpoint, so the API always applies exactly the preprocessing the model was trained with. A resolution of 48×48 keeps digits and pictograms legible while keeping the network small.

6.2 Data Augmentation

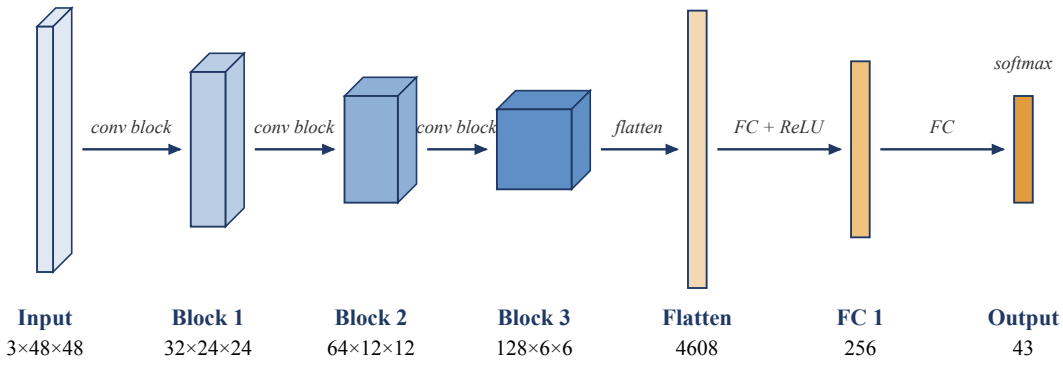
Augmentation is applied only to training images (Table 6.1). Validation and test images receive the deterministic preprocessing only; the training and validation subsets are built from two separate views of the dataset so that validation images are never augmented.

Table 6.1: Training-time data augmentation

Transformation	Parameters	Reason
RandomAffine	rotation ±10°, translation ±10%, scale 0.9–1.1, shear 5°	Camera angle, distance and imperfect crops
ColorJitter	brightness 0.4, contrast 0.4, saturation 0.3	Day/night lighting, shadows, faded paint
Horizontal flip (not used)	n/a	Would turn ‘Keep left’ into ‘Keep right’ and ‘Dangerous curve to the left’ into ‘to the right’, silently corrupting labels

6.3 SignNet Architecture

SignNet is a VGG-style CNN [5] with three convolutional blocks followed by a two-layer classifier (Figure 6.1). Each block applies two 3×3 convolutions with padding 1, each followed by batch normalisation [6] and ReLU, then a 2×2 max-pooling layer that halves the spatial resolution and a dropout layer [7]. The number of filters doubles from block to block (32, 64, 128) while the resolution halves (48, 24, 12, 6), and the dropout rate increases with depth (0.1, 0.2, 0.3). Convolutions omit the bias term because the following batch normalisation has its own shift parameter.



conv block = 2 × [Conv 3×3 (padding 1) → BatchNorm → ReLU] → MaxPool 2×2 → Dropout (0.1, 0.2, 0.3 for blocks 1–3)
FC 1 is followed by Dropout 0.5; the output layer produces 43 logits

Figure 6.1: *SignNet architecture: feature maps after each stage*

The layer-by-layer shapes and parameter counts in Table 6.2 were computed directly from the PyTorch model. In total the network has 1,478,411 trainable parameters, and the checkpoint occupies 5.9 MB. Four fifths of the parameters sit in the first fully connected layer, which maps the 4,608 flattened features to 256 hidden units; this is why it receives the strongest dropout (0.5).

Table 6.2: SignNet layer summary

Stage	Layers	Output shape	Parameters
Input	RGB image, resized and normalised	3 × 48 × 48	0
Block 1	2 × (Conv 3×3, 32 filters, BN, ReLU), MaxPool 2×2, Dropout 0.1	32 × 24 × 24	10,208
Block 2	2 × (Conv 3×3, 64 filters, BN, ReLU), MaxPool 2×2, Dropout 0.2	64 × 12 × 12	55,552
Block 3	2 × (Conv 3×3, 128 filters, BN, ReLU), MaxPool 2×2, Dropout 0.3	128 × 6 × 6	221,696
Flatten	Reshape feature maps to a vector	4,608	0
FC 1	Linear 4,608 → 256, ReLU, Dropout 0.5	256	1,179,904
FC 2	Linear 256 → 43 (logits); softmax at inference	43	11,051
Total			**1,478,411**

6.4 Training Procedure

The network is trained from random initialisation by minimising the cross-entropy loss between the predicted class distribution and the true label. For a mini-batch of N images with true classes y_i, where $p_i(c)$ is the predicted probability of class c for image i, the loss is

$$L = -(1/N) \Sigma_i \log p_i(y_i)$$

AdamW [8] updates the weights with decoupled weight decay, and the learning rate follows a cosine curve from its initial value towards zero over the 15 epochs [9]:

$$\eta_t = \eta_0 \cdot \tfrac{1}{2}\,(1 + \cos(\pi\, t / T)),\ \text{with}\ \eta_0 = 10^{-3}\ \text{and}\ T = 15$$

After every epoch the model is evaluated on the validation subset, and the checkpoint is overwritten only when validation accuracy improves. The checkpoint stores the weights together with the class names, image size and normalisation statistics, and it is loaded with `torch.load(weights_only=True)` so that no arbitrary Python objects are unpickled. Table 6.3 summarises the configuration.

Table 6.3: Training configuration

Setting	Value
Loss function	Cross-entropy
Optimiser	AdamW, initial learning rate 1×10^{-3}, weight decay 1×10^{-4}
Learning-rate schedule	Cosine annealing over 15 epochs ($T_{max} = 15$)
Batch size / epochs	128 / 15
Random seed	42 (data split and weight initialisation)
Model selection	Best validation accuracy
Hardware	Apple M3 GPU (MPS), two data-loader worker processes
Training time	About 8 minutes (30–40 s per epoch)

6.5 Evaluation Metrics

Besides overall accuracy, the model is evaluated per class. For class c, with TP_c, FP_c and FN_c the true positives, false positives and false negatives on the test set:

$$\text{Precision}_c = TP_c / (TP_c + FP_c),\ \text{Recall}_c = TP_c / (TP_c + FN_c)$$

$$F1_c = 2 \cdot \text{Precision}_c \cdot \text{Recall}_c / (\text{Precision}_c + \text{Recall}_c),\ \text{Macro F1} = (1/43)\ \Sigma_c\, F1_c$$

The macro F1 score gives every class the same weight regardless of its size, so it exposes weaknesses on rare classes that overall accuracy would hide. A row-normalised confusion matrix shows which classes are mistaken for which.

6.6 Backend Implementation

The backend (`backend/main.py`) is a FastAPI application. A lifespan handler creates a single `SignClassifier` when the server starts; it selects the device (CUDA, MPS or CPU), loads the checkpoint, builds the preprocessing transform and runs one warm-up prediction so the first real request is not slowed down by GPU initialisation. The checkpoint path can be overridden with the `MODEL_PATH` environment variable and the threshold with `CONFIDENCE_THRESHOLD`. Because model inference is CPU/GPU-bound, it is executed with `run_in_threadpool`, keeping the asynchronous event loop free to accept other requests. All responses are described by Pydantic models, which also generate interactive OpenAPI documentation at `/docs`. The endpoints are listed in Table 6.4.

Table 6.4: REST API endpoints

Method and path	Purpose	Response model
GET /health	Liveness check: status, model loaded, device	HealthResponse
GET /classes	All 43 classes with their category	list[ClassInfo]
GET /model/info	Architecture, parameter count, image size, validation accuracy, test metrics	ModelInfo
POST /predict?top_k=1..10	Classify one image (default top-5)	Prediction
POST /predict/batch?top_k=1..10	Classify up to 32 images in one forward pass (default top-3)	BatchPrediction

Invalid requests are rejected early: an empty upload or a file that cannot be decoded returns HTTP 400, a file larger than 5 MB returns HTTP 413, a batch of more than 32 images returns HTTP 400, and an out-of-range `top_k` returns HTTP 422 through FastAPI's query validation. A typical response for the Stop-sign sample, abridged to the top three classes, is shown below.

```
{
  "filename": "028_class14.png",
  "class_id": 14,
  "label": "Stop",
  "confidence": 0.9992,
  "is_confident": true,
  "top_k": [
    {"class_id": 14, "label": "Stop", "confidence": 0.9992},
    {"class_id": 18, "label": "General caution", "confidence": 0.0004},
    {"class_id": 17, "label": "No entry", "confidence": 0.0003}
  ],
  "inference_ms": 3.4,
  "category": "stop",
  "action": "Come to a full stop at the line, proceed when clear",
  "speed_limit_kmh": null
}
```

6.7 Frontend Implementation

The frontend (`frontend/app.py`) is a Streamlit application. The sidebar holds the API URL and the number of alternatives to show, and it checks the API's health on every run; if the API cannot be reached, the app shows the command needed to start it instead of failing with a stack trace. The main area has four tabs:

- **Classify**: an image from an upload, the bundled test samples or the webcam is sent to the API. The app shows the predicted sign, the confidence, the vehicle action and a top-k table with confidence bars, and for test samples it also shows the ground truth. Uncertain predictions are shown as a warning and the action is replaced with 'keep current behaviour until the sign is confirmed'.
- **Batch**: up to 32 uploaded images, or 12 random test samples, are classified in one request and shown as an image grid and a results table with correctness.
- **Drive simulation**: a simulated car passes a random sequence of 5 to 40 signs. Each frame is classified, the rule-based planner updates the speed limit and commanded speed, and at the end the app shows route accuracy, a speed/limit chart and a log of every decision.
- **Model report**: architecture, parameter count, validation and test accuracy, training curves, the confusion matrix and the hardest classes, all fetched from the API.

The planner (`plan()`, Appendix A.6) starts at 50 km/h. A confident speed-limit sign sets a new limit and the end-of-limit signs restore 100 km/h. The commanded speed is the limit multiplied by a category factor: 0 for stop and no-entry, 0.5 for yield, 0.7 for warnings and 1 otherwise. An uncertain prediction leaves both limit and speed unchanged, which is the safe default for a real vehicle.

CHAPTER 7
Results and Discussion

7.1 Training Behaviour

Figure 7.1 shows the loss and accuracy curves over the 15 epochs and Table 7.1 lists the values. The model learns quickly: validation accuracy passes 96% after four epochs and 99% after seven, and reaches its best value of 99.96% at epoch 12, which is the checkpoint that is deployed. The cosine schedule makes the last epochs smooth, with almost no fluctuation.

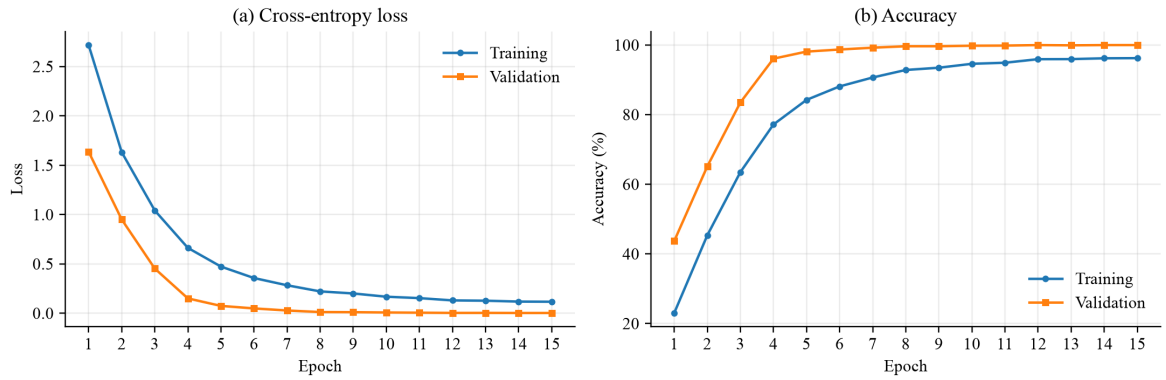


Figure 7.1: *Training and validation loss (a) and accuracy (b) per epoch*

Table 7.1: Per-epoch training history (best epoch 12 in bold)

Epoch	Training loss	Training accuracy	Validation loss	Validation accuracy
1	2.7178	23.02%	1.6368	43.69%
2	1.6320	45.34%	0.9504	65.13%
3	1.0413	63.48%	0.4525	83.48%
4	0.6623	77.16%	0.1504	96.06%
5	0.4729	84.22%	0.0744	98.09%
6	0.3567	88.09%	0.0489	98.69%
7	0.2833	90.65%	0.0279	99.21%
8	0.2213	92.81%	0.0121	99.62%
9	0.2004	93.46%	0.0109	99.62%
10	0.1676	94.59%	0.0077	99.77%
11	0.1532	94.89%	0.0055	99.81%
12	**0.1300**	**95.91%**	**0.0033**	**99.96%**
13	0.1271	95.93%	0.0035	99.89%
14	0.1181	96.17%	0.0026	99.96%
15	0.1167	96.21%	0.0026	99.96%

Validation accuracy is higher than training accuracy throughout. This is expected for two reasons: the training images are augmented and pass through active dropout layers, which makes them harder, while validation images are clean and evaluated with dropout disabled; and the validation subset shares tracks with the training subset (Section 4.4). The gap is therefore not a sign of a problem, but it is the reason the test set is used for all final numbers.

7.2 Test-Set Performance

On the 12,630 images of the official GTSRB test set, which the model never saw during training or model selection, SignNet classifies 12,479 images correctly (Table 7.2).

Table 7.2: Summary of test-set results

Metric	Value
Test accuracy	98.80% (12,479 of 12,630 images)
Misclassified test images	151
Macro-averaged F1 score	0.9821
Classes with F1 = 1.000	3 (classes 32, 33, 34)
Classes with F1 ≥ 0.98	31 of 43
Best validation accuracy	99.96% (epoch 12)
Parameters / checkpoint size	1,478,411 / 5.9 MB

The accuracy of 98.80% exceeds the objective of 97% and is on par with the average human accuracy of 98.84% reported on the same test set [2]. It is below the best published results of 99.46% to 99.71% [3], [10], which rely on committees of networks or spatial transformer modules; SignNet is a single, plain network trained in about eight minutes on a laptop.

7.3 Per-Class Analysis

Figure 7.2 shows the F1 score of every class. Most classes are above 0.98, and only five fall below 0.96. These five classes, listed in Table 7.3, are all among the rarer classes in the training data, with between 150 and 270 training images each compared with a median of 420. Class imbalance is therefore the most likely cause of the remaining errors.

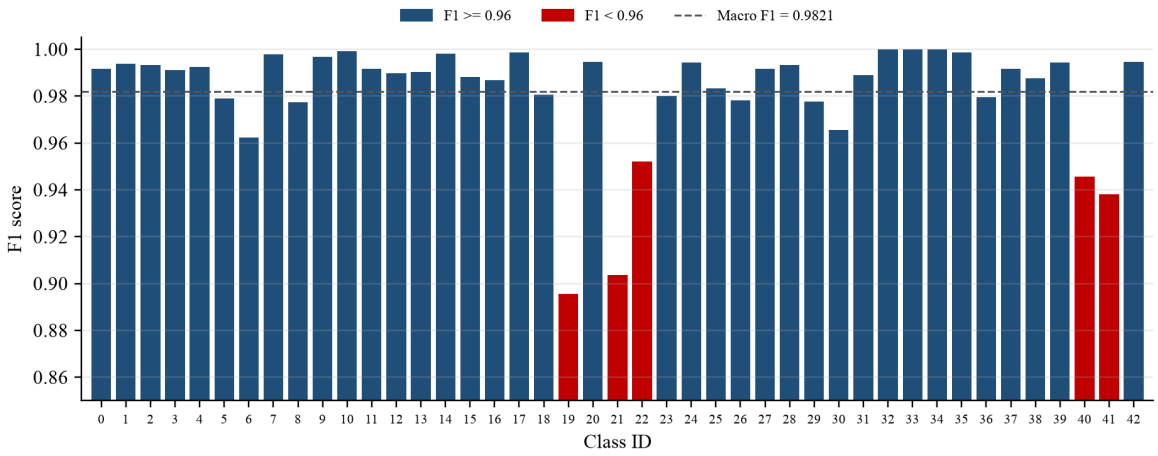


Figure 7.2: *Per-class F1 score on the test set (red: F1 below 0.96)*

Table 7.3: The five weakest classes

ID	Class	Test F1	Training images	Main confusion
19	Dangerous curve to the left	0.896	150	Absorbs Double curve errors (same triangle, similar curve)
21	Double curve	0.904	240	Predicted as Dangerous curve to the left
41	End of no passing	0.938	180	Predicted as No passing (same sign without the stripe)
40	Roundabout mandatory	0.946	240	Confused with other blue circular mandatory signs
22	Bumpy road	0.952	270	Predicted as Road work (dark pictogram at the bottom)

7.4 Confusion Analysis

The normalised confusion matrix in Figure 7.3 is almost perfectly diagonal: for every class, the large majority of test images are predicted correctly, and off-diagonal entries are sparse. Table 7.4 lists the most frequent individual confusions.

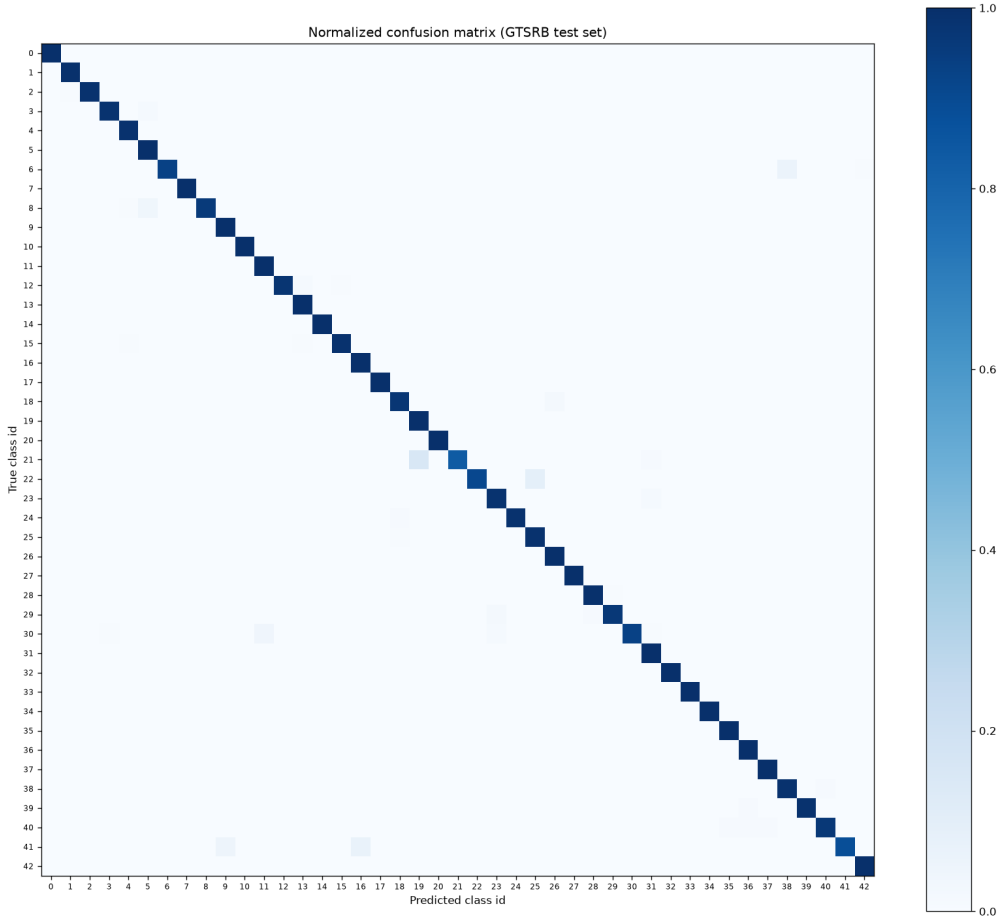


Figure 7.3: *Row-normalised confusion matrix on the GTSRB test set (rows: true class, columns: predicted class)*

Table 7.4: Most frequent confusions on the test set

True class	Predicted as	Images	Likely reason
Speed limit (120km/h) (8)	Speed limit (80km/h) (5)	16	Same red circle; at 48×48 the digits '120' and '80' blur together
Double curve (21)	Dangerous curve to the left (19)	14	Same warning triangle; the pictograms differ by a single bend
Bumpy road (22)	Road work (25)	11	Small dark pictogram at the bottom of a warning triangle
Priority road (12)	Yield (13)	9	Very dark or blurred frames in which the shape cue is lost
End of speed limit (80km/h) (6)	Keep right (38)	9	Both have a diagonal element across a round sign
General caution (18)	Traffic signals (26)	8	Both show a vertical pictogram in a warning triangle
Keep right (38)	Roundabout mandatory (40)	6	Both are blue circular mandatory signs with white arrows
Beware of ice/snow (30)	Right-of-way at the next intersection (11)	6	Small, detailed central symbols (snowflake vs. junction)

The most common error, 120 km/h read as 80 km/h, is also the most safety-relevant, since the vehicle would drive slower than allowed rather than faster; the reverse error would be more dangerous. Most of the other confusions are between signs with the same shape and colour that differ only in a small pictogram, where the 48×48 input leaves very few pixels for the distinguishing detail. A higher input resolution and class-balanced sampling are the most promising remedies (Section 9.3).

7.5 API and Inference Performance

The 86 demo images exported to `data/samples` (two per class, drawn from the test set with a fixed seed) were sent one at a time through the API with the trained model on the Apple M3 GPU (Table 7.5). The timings are the model time measured inside the endpoint, including preprocessing, and exclude network transfer.

Table 7.5: API accuracy and latency on the demo images

Measurement	Result
Demo images classified correctly	85 of 86
Misclassified image	082_class41.png: End of no passing predicted as No passing at 53.4% confidence, flagged as uncertain
Single-image latency (median)	3.31 ms
Single-image latency (95th percentile)	6.04 ms
Single-image latency (maximum)	7.09 ms
Batch of 12 images (total)	382 ms
Model load and warm-up at start-up	a few seconds, once per server start

A median latency of a few milliseconds is far below the 50 ms target, so the model could keep up with a camera running at well over 30 frames per second. The first request after the server has been idle can take tens to a few hundred milliseconds while the GPU wakes up, which is visible in some of the screenshots in the next section; subsequent requests return to the figures above. The one misclassified demo image shows the confidence threshold working as intended: the model hesitated between 'No passing' (53%) and 'Vehicles over 3.5 t prohibited' (41%), the API marked the prediction as not confident, and the frontend told the vehicle to keep its current behaviour instead of acting on the wrong sign.

7.6 Application Demonstration

Figures 7.4 to 7.8 show the running application. In the Classify tab (Figure 7.4) the Stop-sign sample is recognised with 99.9% confidence, and the vehicle action and top-5 alternatives are shown. Figure 7.5 shows the low-confidence case discussed above.

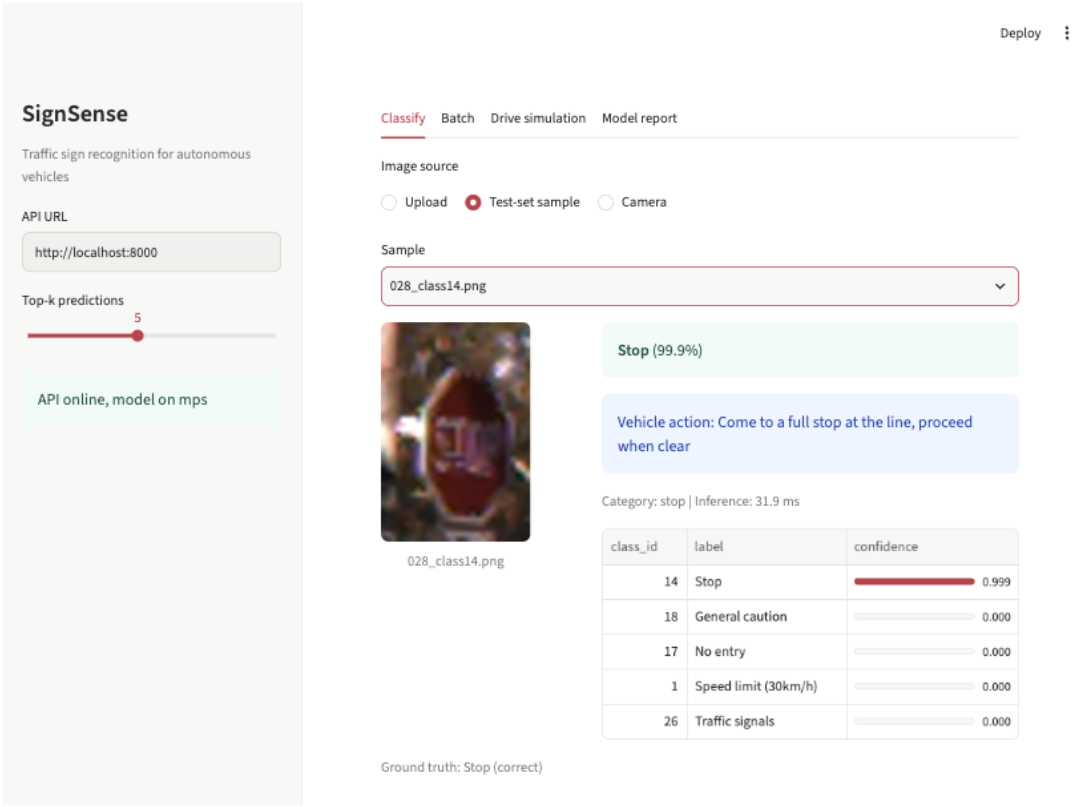


Figure 7.4: *Classify tab: a Stop sign recognised with 99.9% confidence*

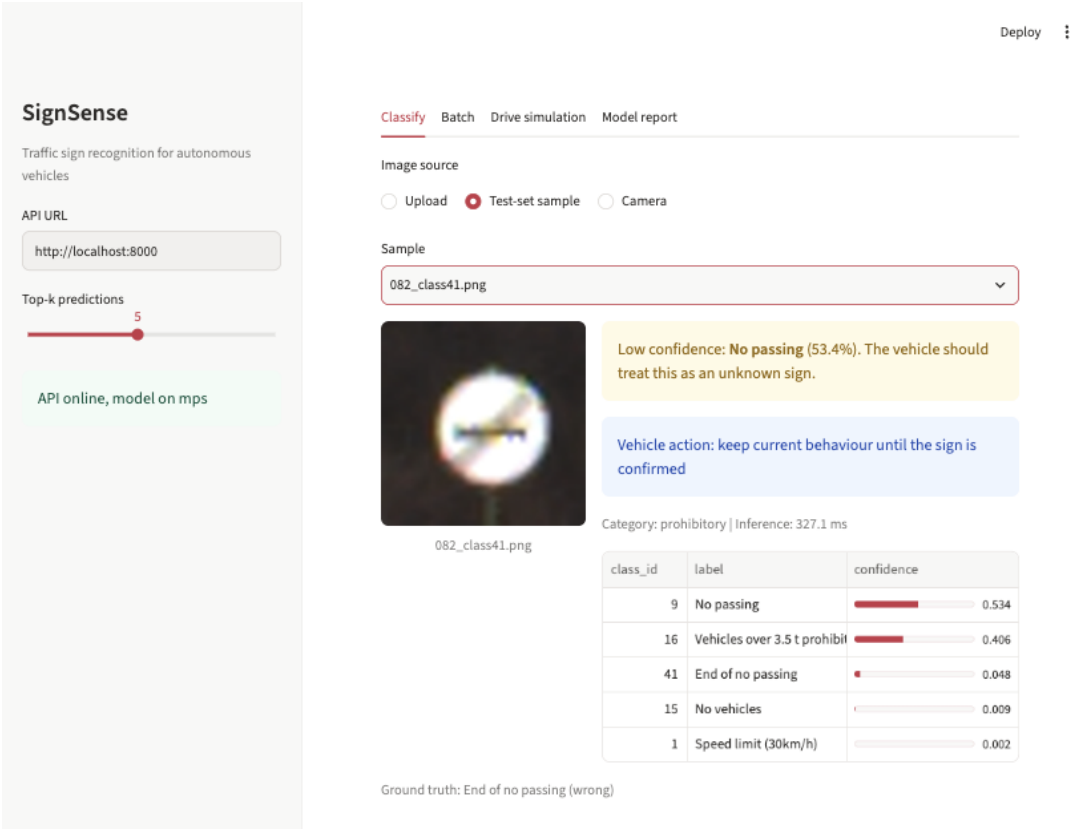


Figure 7.5: *Low-confidence prediction: the action is withheld until the sign is confirmed*

The Batch tab (Figure 7.6) classifies 12 random samples in a single request. The confidence under each image makes the harder cases stand out, such as the ‘End of no passing’ sample at 58%.

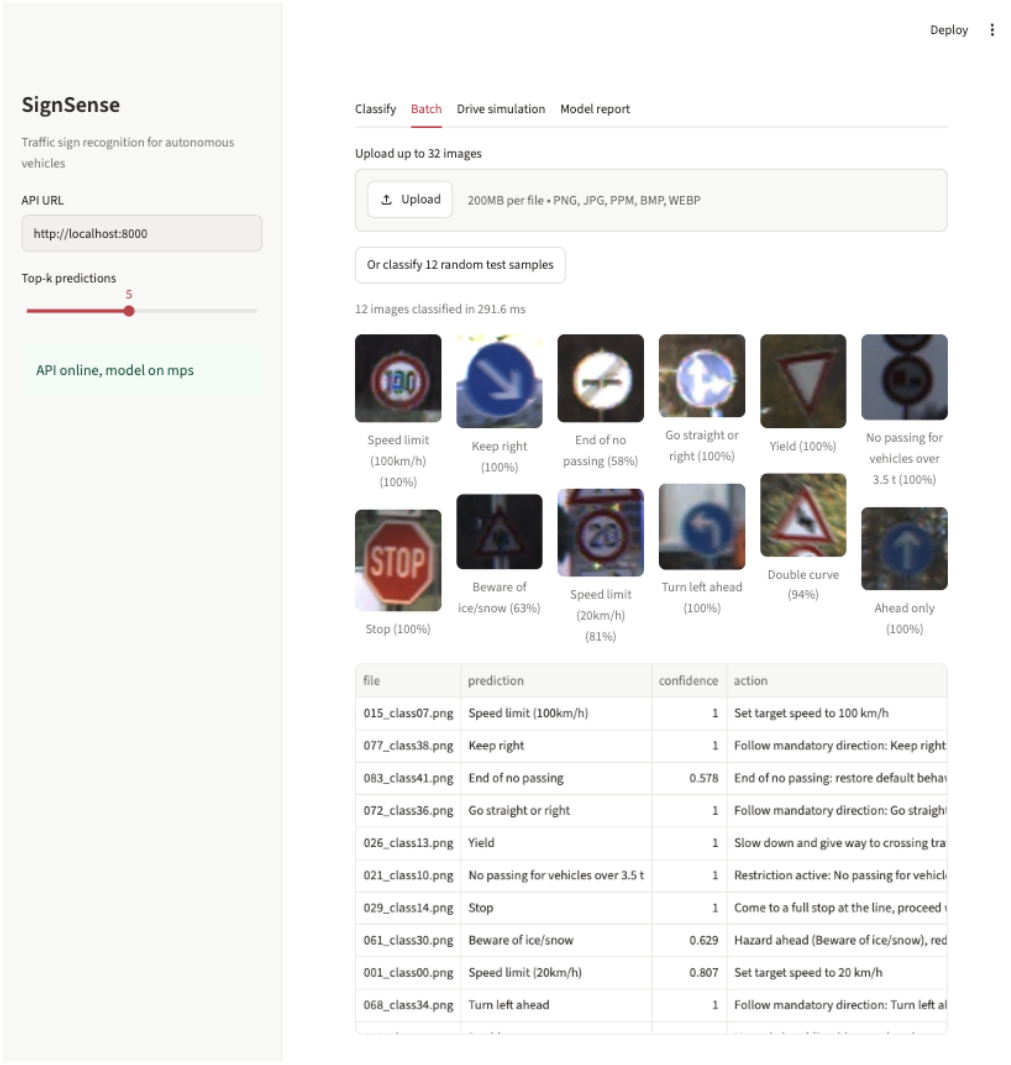


Figure 7.6: *Batch tab: 12 random test samples classified in one request*

In the drive simulation (Figure 7.7) the planner reacts to each sign: the speed drops to 35 km/h (0.7 × 50) at a warning sign, to zero at a no-entry sign, and the limit rises to 100 km/h at step 9. All 12 signs on the route shown were classified correctly; in an earlier recorded run, 11 of 12 were correct (92%).

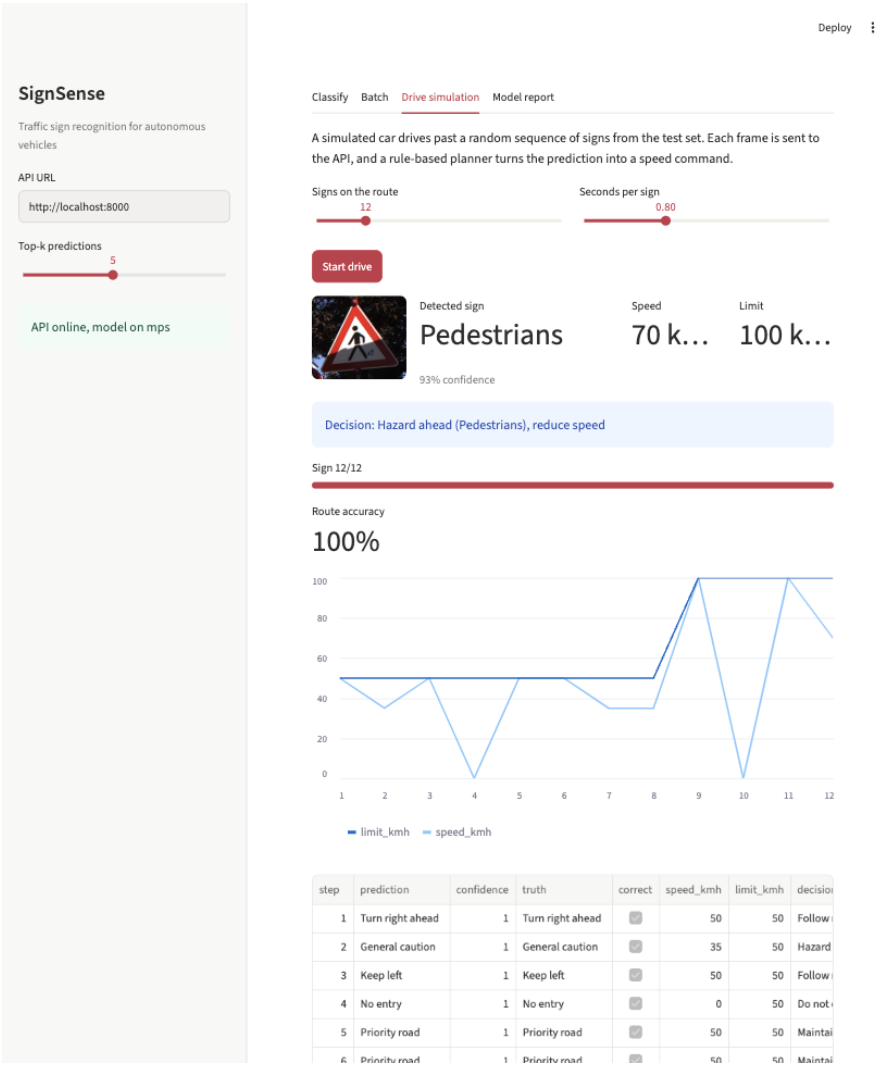


Figure 7.7: *Drive simulation: the planner's speed and limit over a 12-sign route*

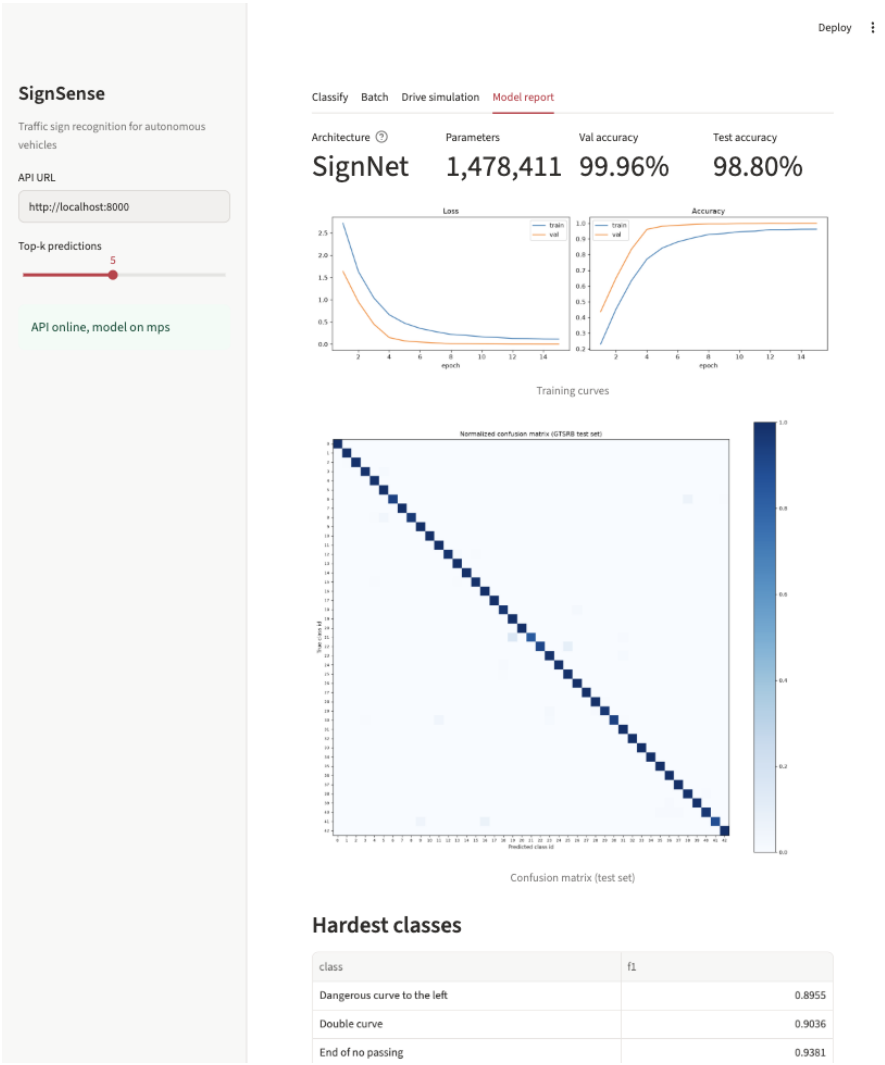


Figure 7.8: *Model report tab with headline metrics, curves and the confusion matrix*

7.7 Discussion

The results show that a compact, single CNN trained from scratch is sufficient for accurate traffic sign classification on GTSRB, reaching human-level accuracy with a model that trains in minutes and runs in milliseconds. The main value of the project, however, lies in the complete pipeline: careful augmentation choices that respect sign semantics, an evaluation that looks beyond accuracy, and a service that tells the rest of the system when not to trust the model. The error analysis also makes clear where the model would need to improve before it could be relied upon: rare warning signs with small pictograms, and speed limits whose digits are hard to read at low resolution.

CHAPTER 8
Testing and Validation

8.1 Automated Tests

The project has 21 automated tests written with pytest, located in `tests/`. API tests use FastAPI's `TestClient` with a freshly initialised, untrained SignNet saved to a temporary checkpoint and forced onto the CPU, so they run in about three seconds without the real model or a GPU. Machine-learning tests check the model and the sign semantics, and two of them load the trained checkpoint to confirm that known demo images are still classified correctly. Table 8.1 lists the tests; all 21 pass.

Table 8.1: Automated test suite

Test	What it verifies	Cases
test_health	Health endpoint reports status ok, model loaded and the device	1
test_classes	43 classes are returned; class 14 is 'Stop' with category stop	1
test_model_info	Model metadata: 43 classes, image size, parameter count	1
test_predict_returns_ranked_top_k	top-k has the requested length, is sorted and matches the label	1
test_untrained_model_is_not_confident	An untrained model is flagged as not confident	1
test_predict_rejects_non_image	A text file is rejected with HTTP 400	1
test_predict_rejects_empty_file	An empty upload is rejected with HTTP 400	1
test_predict_rejects_out_of_range_top_k	top_k of 0 or 11 is rejected with HTTP 422	2
test_batch	Batch predictions keep the order of the uploaded files	1
test_batch_rejects_more_than_32_images	A batch of 33 images is rejected with HTTP 400	1
test_model_output_shape	SignNet maps a 3×48×48 batch to 43 logits	1
test_every_class_has_a_known_category	Every class maps to one of the nine categories	1
test_speed_limits	Speed-limit classes return the correct limit (20, 30, 50, 100, 120)	5
test_only_speed_limit_signs_set_a_speed	Only speed-limit signs carry a speed value	1
test_trained_model_on_demo_samples	Trained model predicts Stop and 50 km/h samples with confidence above 0.9	2
Total		**21**

8.2 System-Level Validation

In addition to the unit and integration tests, the complete system was exercised manually with the backend and frontend running together (Table 8.2).

Table 8.2: System-level validation scenarios

Scenario	Expected behaviour	Result
Classify a clear sign from the test samples	Correct label, high confidence, action shown	Pass
Classify an ambiguous sign (082_class41)	Warning shown, action withheld	Pass
Batch of 12 random samples	Grid and table, correctness per image	Pass
Drive simulation with 12 signs	Speed follows the planner rules, summary chart and log	Pass
Model report tab	Metrics, curves and confusion matrix load from the API	Pass
All 86 demo images through the API	85 correct, the error flagged as uncertain	Pass
Upload larger than 5 MB	HTTP 413 with an explanatory message	Pass

CHAPTER 9
Conclusion and Future Scope

9.1 Conclusion

This project set out to build a complete traffic sign recognition system for autonomous vehicle simulations, from raw data to an interactive demonstration, and all objectives were met. A compact VGG-style CNN with 1.48 million parameters, trained from scratch on GTSRB in about eight minutes on a laptop GPU, reaches 98.80% accuracy and a macro F1 score of 0.9821 on the official test set, on par with reported human performance. The model is served through a validated FastAPI service with a median inference time of a few milliseconds, and a Streamlit application lets a user classify images, inspect the model and watch a simulated vehicle react to the signs it sees.

Beyond accuracy, the project shows several practices that matter for safety-related machine learning: choosing augmentations that do not change the meaning of the data, evaluating on a truly held-out test set with per-class metrics, analysing the errors that remain, and making the system aware of its own uncertainty so that it can decline to act on a doubtful prediction.

9.2 Limitations

- **Classification only:** the system expects an image containing a single sign and cannot yet find signs inside full camera frames.
- **German signs only:** GTSRB covers German signs. Indian road signs differ in design and include classes that GTSRB lacks, so the model would need to be retrained or fine-tuned for Indian roads.
- **Single-frame decisions:** each image is classified independently, although a real vehicle sees the same sign in many consecutive frames.
- **Uncalibrated confidence:** softmax probabilities are not guaranteed to match true correctness rates, so the 0.6 threshold was chosen by inspection rather than calibrated.
- **Class imbalance:** no re-weighting or balanced sampling was used, and the weakest classes are among the rarest.
- **Simplified planner:** the rule-based planner illustrates the decision path but is not a vehicle controller and ignores other traffic, the road layout and the vehicle's dynamics.

9.3 Future Scope

- **Detection stage:** add a detector such as a YOLO-family model trained on GTSDB [20] to locate signs in full dash-camera frames, and feed the crops to SignNet.
- **Temporal smoothing:** combine predictions over consecutive frames (for example by voting or averaging probabilities) before the planner acts.
- **Accuracy improvements:** use a 64×64 input, class-balanced sampling or focal loss, spatial transformer layers [11] or an ensemble to close the gap to the best published results.
- **Calibration and explainability:** apply temperature scaling so the threshold has a probabilistic meaning, and show Grad-CAM [19] heat maps in the interface to explain each prediction.
- **Indian traffic signs:** collect or adopt a dataset of Indian signs and fine-tune the model with transfer learning.
- **Simulator integration:** connect the API to an open driving simulator such as CARLA [16] for closed-loop testing.
- **Edge deployment:** export the model to TorchScript or ONNX, quantise it and package the services with Docker for embedded hardware.

REFERENCES

[1] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "The German Traffic Sign Recognition Benchmark: A multi-class classification competition," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2011, pp. 1453–1460.

[2] J. Stallkamp, M. Schlipsing, J. Salmen and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural Networks*, vol. 32, pp. 323–332, 2012.

[3] D. Cireşan, U. Meier, J. Masci and J. Schmidhuber, "Multi-column deep neural network for traffic sign classification," *Neural Networks*, vol. 32, pp. 333–338, 2012.

[4] P. Sermanet and Y. LeCun, "Traffic sign recognition with multi-scale convolutional networks," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2011, pp. 2809–2813.

[5] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," in *Proc. International Conference on Learning Representations (ICLR)*, 2015.

[6] S. Ioffe and C. Szegedy, "Batch normalization: Accelerating deep network training by reducing internal covariate shift," in *Proc. International Conference on Machine Learning (ICML)*, 2015, pp. 448–456.

[7] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A simple way to prevent neural networks from overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929–1958, 2014.

[8] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[9] I. Loshchilov and F. Hutter, "SGDR: Stochastic gradient descent with warm restarts," in *Proc. International Conference on Learning Representations (ICLR)*, 2017.

[10] Á. Arcos-García, J. A. Álvarez-García and L. M. Soria-Morillo, "Deep neural network for traffic sign recognition systems: An analysis of spatial transformers and stochastic optimisation methods," *Neural Networks*, vol. 99, pp. 158–165, 2018.

[11] M. Jaderberg, K. Simonyan, A. Zisserman and K. Kavukcuoglu, "Spatial transformer networks," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.

[12] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

[13] Y. LeCun, L. Bottou, Y. Bengio and P. Haffner, "Gradient-based learning applied to document recognition," *Proceedings of the IEEE*, vol. 86, no. 11, pp. 2278–2324, 1998.

[14] N. Dalal and B. Triggs, "Histograms of oriented gradients for human detection," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886–893.

[15] SAE International, "Taxonomy and definitions for terms related to driving automation systems for on-road motor vehicles," Standard J3016, 2021.

[16] A. Dosovitskiy, G. Ros, F. Codevilla, A. López and V. Koltun, "CARLA: An open urban driving simulator," in *Proc. Conference on Robot Learning (CoRL)*, 2017.

[17] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[18] Streamlit Inc., "Streamlit documentation." [Online]. Available: https://docs.streamlit.io

[19] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proc. IEEE International Conference on Computer Vision (ICCV)*, 2017.

[20] S. Houben, J. Stallkamp, J. Salmen, M. Schlipsing and C. Igel, "Detection of traffic signs in real-world images: The German Traffic Sign Detection Benchmark," in *Proc. International Joint Conference on Neural Networks (IJCNN)*, 2013.

[21] V. A. Singh, "SignSense: Traffic sign recognition for autonomous vehicles," source code repository, 2026. [Online]. Available: https://github.com/Vaibhavanandsingh/sign_sense

APPENDIX A
Source Code Listings

This appendix reproduces the key source files of the project. The complete code, including the evaluation script, Pydantic schemas, tests and documentation, is available at https://github.com/Vaibhavanandsingh/sign_sense. Line numbers match the files in the repository.

A.1 SignNet model

File: `ml/model.py` (35 lines)

```
from torch import nn


def conv_block(in_ch: int, out_ch: int, dropout: float) -> nn.Sequential:
    return nn.Sequential(
        nn.Conv2d(in_ch, out_ch, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.Conv2d(out_ch, out_ch, kernel_size=3, padding=1, bias=False),
        nn.BatchNorm2d(out_ch),
        nn.ReLU(inplace=True),
        nn.MaxPool2d(2),
        nn.Dropout(dropout),
    )


class SignNet(nn.Module):
    def __init__(self, num_classes: int = 43, img_size: int = 48):
        super().__init__()
        self.features = nn.Sequential(
            conv_block(3, 32, 0.1),
            conv_block(32, 64, 0.2),
            conv_block(64, 128, 0.3),
        )
        feature_size = img_size // 8
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * feature_size * feature_size, 256),
            nn.ReLU(inplace=True),
            nn.Dropout(0.5),
            nn.Linear(256, num_classes),
        )

    def forward(self, x):
        return self.classifier(self.features(x))
```

A.2 Data pipeline and augmentation

File: `ml/data.py` (50 lines)

```
import torch
from torch.utils.data import DataLoader, Subset
from torchvision import datasets, transforms

```

```
from ml.config import DATA_DIR, IMG_SIZE, MEAN, STD


def build_transform(train: bool) -> transforms.Compose:
    steps = [transforms.Resize((IMG_SIZE, IMG_SIZE))]
    if train:
        steps += [
            transforms.RandomAffine(degrees=10, translate=(0.1, 0.1), scale=(0.9, 1.1),
        shear=5),
            transforms.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.3),
        ]
    steps += [transforms.ToTensor(), transforms.Normalize(MEAN, STD)]
    return transforms.Compose(steps)


def _loader(dataset, batch_size: int, shuffle: bool, num_workers: int) -> DataLoader:
    return DataLoader(
        dataset,
        batch_size=batch_size,
        shuffle=shuffle,
        num_workers=num_workers,
        persistent_workers=num_workers > 0,
        pin_memory=torch.cuda.is_available(),
    )


def get_dataloaders(batch_size: int = 128, val_fraction: float = 0.1,
                    num_workers: int = 2, seed: int = 42) -> tuple[DataLoader,
        DataLoader]:
    # Two views of the same training set so validation images are not augmented.
    train_view = datasets.GTSRB(DATA_DIR, split="train", transform=build_transform(True),
        download=True)
    val_view = datasets.GTSRB(DATA_DIR, split="train", transform=build_transform(False),
        download=True)

    generator = torch.Generator().manual_seed(seed)
    indices = torch.randperm(len(train_view), generator=generator).tolist()
    n_val = int(len(indices) * val_fraction)

    train_ds = Subset(train_view, indices[n_val:])
    val_ds = Subset(val_view, indices[:n_val])
    return (
        _loader(train_ds, batch_size, True, num_workers),
        _loader(val_ds, batch_size, False, num_workers),
    )


def get_test_loader(batch_size: int = 256, num_workers: int = 2) -> DataLoader:
    test_ds = datasets.GTSRB(DATA_DIR, split="test", transform=build_transform(False),
        download=True)
    return _loader(test_ds, batch_size, False, num_workers)
```

A.3 Training loop

File: `ml/train.py` (113 lines)

```
import argparse
import json

import matplotlib

matplotlib.use("Agg")
import matplotlib.pyplot as plt
import torch
from torch import nn
from tqdm import tqdm
```

```
11
12  from ml.config import (CHECKPOINT_PATH, CLASS_NAMES, IMG_SIZE, MEAN, MODELS_DIR,
13                         NUM_CLASSES, REPORTS_DIR, STD)
14  from ml.data import get_dataloaders
15  from ml.model import SignNet
16  from ml.utils import get_device
17
18
19  def run_epoch(model, loader, criterion, device, optimizer=None) -> tuple[float, float]:
20      training = optimizer is not None
21      model.train(training)
22      total_loss, correct, seen = 0.0, 0, 0
23      with torch.set_grad_enabled(training):
24          for images, labels in tqdm(loader, leave=False, desc="train" if training else
            "val"):
25              images, labels = images.to(device), labels.to(device)
26              logits = model(images)
27              loss = criterion(logits, labels)
28              if training:
29                  optimizer.zero_grad(set_to_none=True)
30                  loss.backward()
31                  optimizer.step()
32              total_loss += loss.item() * labels.size(0)
33              correct += (logits.argmax(dim=1) == labels).sum().item()
34              seen += labels.size(0)
35      return total_loss / seen, correct / seen
36
37
38  def plot_history(history: dict, path) -> None:
39      epochs = range(1, len(history["train_loss"]) + 1)
40      fig, (ax_loss, ax_acc) = plt.subplots(1, 2, figsize=(12, 4))
41      ax_loss.plot(epochs, history["train_loss"], label="train")
42      ax_loss.plot(epochs, history["val_loss"], label="val")
43      ax_loss.set_title("Loss")
44      ax_loss.set_xlabel("epoch")
45      ax_loss.legend()
46      ax_acc.plot(epochs, history["train_acc"], label="train")
47      ax_acc.plot(epochs, history["val_acc"], label="val")
48      ax_acc.set_title("Accuracy")
49      ax_acc.set_xlabel("epoch")
50      ax_acc.legend()
51      fig.tight_layout()
52      fig.savefig(path, dpi=120)
53      plt.close(fig)
54
55
56  def main() -> None:
57      parser = argparse.ArgumentParser(description="Train SignNet on GTSRB")
58      parser.add_argument("--epochs", type=int, default=15)
59      parser.add_argument("--batch-size", type=int, default=128)
60      parser.add_argument("--lr", type=float, default=1e-3)
61      parser.add_argument("--num-workers", type=int, default=2)
62      args = parser.parse_args()
63
64      torch.manual_seed(42)
65      MODELS_DIR.mkdir(parents=True, exist_ok=True)
66      REPORTS_DIR.mkdir(parents=True, exist_ok=True)
67
68      device = get_device()
69      print(f"Training on {device}")
70      train_loader, val_loader = get_dataloaders(args.batch_size,
            num_workers=args.num_workers)
71
72      model = SignNet(NUM_CLASSES, IMG_SIZE).to(device)
73      criterion = nn.CrossEntropyLoss()
74      optimizer = torch.optim.AdamW(model.parameters(), lr=args.lr, weight_decay=1e-4)
75      scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=args.epochs)
76
77      history = {"train_loss": [], "train_acc": [], "val_loss": [], "val_acc": []}
78      best_acc = 0.0
```

```
    for epoch in range(1, args.epochs + 1):
        train_loss, train_acc = run_epoch(model, train_loader, criterion, device,
        optimizer)
        val_loss, val_acc = run_epoch(model, val_loader, criterion, device)
        scheduler.step()

        history["train_loss"].append(train_loss)
        history["train_acc"].append(train_acc)
        history["val_loss"].append(val_loss)
        history["val_acc"].append(val_acc)
        print(f"epoch {epoch:02d} | train loss {train_loss:.4f} acc {train_acc:.4f} "
              f"| val loss {val_loss:.4f} acc {val_acc:.4f}")

        if val_acc > best_acc:
            best_acc = val_acc
            torch.save(
                {
                    "state_dict": {k: v.cpu() for k, v in model.state_dict().items()},
                    "class_names": CLASS_NAMES,
                    "img_size": IMG_SIZE,
                    "mean": MEAN,
                    "std": STD,
                    "val_acc": val_acc,
                    "epoch": epoch,
                },
                CHECKPOINT_PATH,
            )
            print(f"  saved new best model (val acc {val_acc:.4f})")

    (REPORTS_DIR / "history.json").write_text(json.dumps(history, indent=2))
    plot_history(history, REPORTS_DIR / "training_curves.png")
    print(f"Best val accuracy {best_acc:.4f}, checkpoint at {CHECKPOINT_PATH}")


if __name__ == "__main__":
    main()
```

A.4 Inference wrapper

File: `backend/inference.py` (44 lines)

```
import io
from pathlib import Path

import torch
from PIL import Image
from torchvision import transforms

from ml.model import SignNet
from ml.utils import get_device


class SignClassifier:
    def __init__(self, checkpoint_path: Path):
        self.device = get_device()
        checkpoint = torch.load(checkpoint_path, map_location="cpu", weights_only=True)
        self.class_names: list[str] = list(checkpoint["class_names"])
        self.img_size: int = checkpoint["img_size"]
        self.val_acc: float | None = checkpoint.get("val_acc")

        self.model = SignNet(len(self.class_names), self.img_size)
        self.model.load_state_dict(checkpoint["state_dict"])
        self.model.to(self.device).eval()
        self.num_parameters = sum(p.numel() for p in self.model.parameters())

        self.transform = transforms.Compose([
```

```
            transforms.Resize((self.img_size, self.img_size)),
            transforms.ToTensor(),
            transforms.Normalize(checkpoint["mean"], checkpoint["std"]),
        ])
        self.predict([Image.new("RGB", (self.img_size, self.img_size))])  # warm-up

    @staticmethod
    def load_image(data: bytes) -> Image.Image:
        return Image.open(io.BytesIO(data)).convert("RGB")

    @torch.inference_mode()
    def predict(self, images: list[Image.Image], top_k: int = 5) -> list[list[tuple[int, float]]]:
        batch = torch.stack([self.transform(image) for image in images]).to(self.device)
        probabilities = torch.softmax(self.model(batch), dim=1)
        confidences, indices = probabilities.topk(min(top_k, len(self.class_names)), dim=1)
        return [
            list(zip(idx.tolist(), conf.tolist()))
            for idx, conf in zip(indices.cpu(), confidences.cpu())
        ]
```

A.5 FastAPI backend

File: `backend/main.py` (123 lines)

```
import json
import os
import time
from contextlib import asynccontextmanager
from pathlib import Path

from fastapi import FastAPI, File, HTTPException, Query, UploadFile
from fastapi.concurrency import run_in_threadpool
from fastapi.middleware.cors import CORSMiddleware
from PIL import Image

from backend.inference import SignClassifier
from backend.schemas import (BatchPrediction, ClassInfo, HealthResponse, ModelInfo,
                             Prediction, TopKItem)
from ml.config import CHECKPOINT_PATH, REPORTS_DIR
from ml.signs import describe

CONFIDENCE_THRESHOLD = float(os.getenv("CONFIDENCE_THRESHOLD", "0.6"))
MAX_FILE_BYTES = 5 * 1024 * 1024
MAX_BATCH = 32

state: dict[str, SignClassifier] = {}


@asynccontextmanager
async def lifespan(app: FastAPI):
    checkpoint = Path(os.getenv("MODEL_PATH", CHECKPOINT_PATH))
    if not checkpoint.exists():
        raise RuntimeError(f"No model at {checkpoint}. Run `python -m ml.train` first.")
    state["classifier"] = SignClassifier(checkpoint)
    yield
    state.clear()


app = FastAPI(
    title="SignSense API",
    description="Traffic sign recognition for autonomous vehicle simulations",
    version="0.1.0",
    lifespan=lifespan,
)
```

```
app.add_middleware(CORSMiddleware, allow_origins=["*"], allow_methods=["*"],
        allow_headers=["*"])


def classifier() -> SignClassifier:
    return state["classifier"]


async def read_image(file: UploadFile) -> Image.Image:
    data = await file.read()
    if not data:
        raise HTTPException(400, f"{file.filename} is empty")
    if len(data) > MAX_FILE_BYTES:
        raise HTTPException(413, f"{file.filename} is larger than 5 MB")
    try:
        return SignClassifier.load_image(data)
    except OSError:
        raise HTTPException(400, f"{file.filename} is not a valid image")


def to_prediction(filename: str | None, ranked: list[tuple[int, float]], elapsed_ms:
        float) -> Prediction:
    names = classifier().class_names
    class_id, confidence = ranked[0]
    return Prediction(
        filename=filename,
        class_id=class_id,
        label=names[class_id],
        confidence=round(confidence, 4),
        is_confident=confidence >= CONFIDENCE_THRESHOLD,
        top_k=[TopKItem(class_id=c, label=names[c], confidence=round(p, 4)) for c, p in
        ranked],
        inference_ms=round(elapsed_ms, 2),
        **describe(class_id),
    )


@app.get("/health", response_model=HealthResponse)
def health():
    return HealthResponse(status="ok", model_loaded="classifier" in state,
        device=str(classifier().device))


@app.get("/classes", response_model=list[ClassInfo])
def classes():
    return [
        ClassInfo(class_id=i, label=name, category=describe(i)["category"])
        for i, name in enumerate(classifier().class_names)
    ]


@app.get("/model/info", response_model=ModelInfo)
def model_info():
    clf = classifier()
    metrics_path = REPORTS_DIR / "metrics.json"
    return ModelInfo(
        architecture="SignNet (3-block CNN)",
        num_classes=len(clf.class_names),
        num_parameters=clf.num_parameters,
        img_size=clf.img_size,
        device=str(clf.device),
        val_accuracy=clf.val_acc,
        test_metrics=json.loads(metrics_path.read_text()) if metrics_path.exists() else
        None,
    )


@app.post("/predict", response_model=Prediction)
async def predict(file: UploadFile = File(...), top_k: int = Query(5, ge=1, le=10)):
    image = await read_image(file)
```

```
    start = time.perf_counter()
    ranked = (await run_in_threadpool(classifier().predict, [image], top_k))[0]
    return to_prediction(file.filename, ranked, (time.perf_counter() - start) * 1000)


@app.post("/predict/batch", response_model=BatchPrediction)
async def predict_batch(files: list[UploadFile] = File(...), top_k: int = Query(3, ge=1,
        le=10)):
    if len(files) > MAX_BATCH:
        raise HTTPException(400, f"Send at most {MAX_BATCH} images per batch")
    images = [await read_image(f) for f in files]
    start = time.perf_counter()
    ranked = await run_in_threadpool(classifier().predict, images, top_k)
    total_ms = (time.perf_counter() - start) * 1000
    per_image_ms = total_ms / len(files)
    return BatchPrediction(
        predictions=[to_prediction(f.filename, r, per_image_ms) for f, r in zip(files,
        ranked)],
        total_ms=round(total_ms, 2),
    )
```

A.6 Rule-based planner

File: `frontend/app.py`, function `plan()` (10 lines)

```
def plan(limit: int, result: dict) -> tuple[int, int, str]:
    """Rule-based planner: returns (new speed limit, commanded speed, decision text)."""
    if not result["is_confident"]:
        return limit, limit, f"Uncertain sign ({result['confidence']:.0%}), keeping
        current limit"
    if result["speed_limit_kmh"]:
        limit = result["speed_limit_kmh"]
    elif result["class_id"] in (6, 32):
        limit = OPEN_ROAD_LIMIT
    speed = round(limit * SPEED_FACTOR.get(result["category"], 1.0))
    return limit, speed, result["action"]
```

A.7 Sign semantics

File: `ml/signs.py` (38 lines)

```
from ml.config import CLASS_NAMES

SPEED_LIMITS = {0: 20, 1: 30, 2: 50, 3: 60, 4: 70, 5: 80, 7: 100, 8: 120}
END_OF_RESTRICTION = {6, 32, 41, 42}
NO_ENTRY = {15, 17}
PROHIBITORY = {9, 10, 16}
PRIORITY = {11, 12}
WARNING = set(range(18, 32))
MANDATORY = set(range(33, 41))
YIELD, STOP = 13, 14


def _info(category: str, action: str, speed_limit_kmh: int | None = None) -> dict:
    return {"category": category, "action": action, "speed_limit_kmh": speed_limit_kmh}


def describe(class_id: int) -> dict:
    name = CLASS_NAMES[class_id]
    if class_id in SPEED_LIMITS:
        limit = SPEED_LIMITS[class_id]
        return _info("speed_limit", f"Set target speed to {limit} km/h", limit)
    if class_id == STOP:
```

```
        return _info("stop", "Come to a full stop at the line, proceed when clear")
    if class_id == YIELD:
        return _info("yield", "Slow down and give way to crossing traffic")
    if class_id in PRIORITY:
        return _info("priority", "Maintain speed, you have right of way")
    if class_id in NO_ENTRY:
        return _info("no_entry", "Do not enter, plan an alternative route")
    if class_id in PROHIBITORY:
        return _info("prohibitory", f"Restriction active: {name}")
    if class_id in END_OF_RESTRICTION:
        return _info("end_of_restriction", f"{name}: restore default behaviour")
    if class_id in WARNING:
        return _info("warning", f"Hazard ahead ({name}), reduce speed")
    if class_id in MANDATORY:
        return _info("mandatory", f"Follow mandatory direction: {name}")
    return _info("other", name)
```

APPENDIX B

GTSRB Class Reference

The 43 GTSRB classes with the category used by the planner, the number of training and test images, and the F1 score of SignNet on the test set.

Table B.1: GTSRB classes, categories, image counts and test F1 scores

ID	Class name	Category	Train	Test	F1
0	Speed limit (20km/h)	speed_limit	150	60	0.992
1	Speed limit (30km/h)	speed_limit	1,500	720	0.994
2	Speed limit (50km/h)	speed_limit	1,500	750	0.993
3	Speed limit (60km/h)	speed_limit	960	450	0.991
4	Speed limit (70km/h)	speed_limit	1,320	660	0.992
5	Speed limit (80km/h)	speed_limit	1,260	630	0.979
6	End of speed limit (80km/h)	end_of_restriction	300	150	0.962
7	Speed limit (100km/h)	speed_limit	960	450	0.998
8	Speed limit (120km/h)	speed_limit	960	450	0.977
9	No passing	prohibitory	990	480	0.997
10	No passing for vehicles over 3.5 t	prohibitory	1,350	660	0.999
11	Right-of-way at the next intersection	priority	900	420	0.992
12	Priority road	priority	1,410	690	0.990
13	Yield	yield	1,440	720	0.990
14	Stop	stop	540	270	0.998
15	No vehicles	no_entry	420	210	0.988
16	Vehicles over 3.5 t prohibited	prohibitory	300	150	0.987
17	No entry	no_entry	750	360	0.999
18	General caution	warning	810	390	0.981
19	Dangerous curve to the left	warning	150	60	0.896
20	Dangerous curve to the right	warning	240	90	0.994
21	Double curve	warning	240	90	0.904
22	Bumpy road	warning	270	120	0.952
23	Slippery road	warning	360	150	0.980
24	Road narrows on the right	warning	180	90	0.994
25	Road work	warning	1,020	480	0.983
26	Traffic signals	warning	420	180	0.978
27	Pedestrians	warning	180	60	0.992
28	Children crossing	warning	360	150	0.993

ID	Class name	Category	Train	Test	F1
29	Bicycles crossing	warning	180	90	0.978
30	Beware of ice/snow	warning	300	150	0.966
31	Wild animals crossing	warning	540	270	0.989
32	End of all speed and passing limits	end_of_restriction	180	60	1.000
33	Turn right ahead	mandatory	480	210	1.000
34	Turn left ahead	mandatory	300	120	1.000
35	Ahead only	mandatory	810	390	0.999
36	Go straight or right	mandatory	270	120	0.980
37	Go straight or left	mandatory	150	60	0.992
38	Keep right	mandatory	1,380	690	0.988
39	Keep left	mandatory	210	90	0.994
40	Roundabout mandatory	mandatory	240	90	0.946
41	End of no passing	end_of_restriction	180	60	0.938
42	End of no passing for vehicles over 3.5 t	end_of_restriction	180	90	0.994